



JavaScript中级-API

API

001.API简介:

API是应用程序编程接口，可以扩展浏览器的功能、简化复杂的功能、为复杂代码提供简单语法。

Web API适用于Web的应用程序编程接口

浏览器API和服务端API可以扩展Web浏览器的功能

浏览器内置API与第三方API

002.WebAPI访问表单form:

约束验证DOM方法: `checkValidity ()` 与 `setCustomValidity ()`

如果输入字段包含无效数据，则显示一条消息

约束验证DOM属性: `validity`、`validationMessage`、`willValidate`

表单验证的检验状态`validity`的属性，用于检查表单输入是否符合预定义的规则

这些属性通常用于前端表单验证，帮助开发者在用户提交表单之前检查输入值是否符合预期，从而提供即时反馈并防止无效数据提交到服务器。

属性	描述
<code>customError</code>	如果设置了自定义有效性消息，则设置为 <code>true</code> 。
<code>patternMismatch</code>	如果元素的值与其 <code>pattern</code> 属性不匹配，则设置为 <code>true</code> 。
<code>rangeOverflow</code>	如果元素的值大于其 <code>max</code> 属性，则设置为 <code>true</code> 。
<code>rangeUnderflow</code>	如果元素的值小于其 <code>min</code> 属性，则设置为 <code>true</code> 。
<code>stepMismatch</code>	如果元素的值对其 <code>step</code> 属性无效，则设置为 <code>true</code> 。
<code>tooLong</code>	如果元素的值超过其 <code>maxLength</code> 属性，则设置为 <code>true</code> 。
<code>typeMismatch</code>	如果元素的值对其 <code>type</code> 属性无效，则设置为 <code>true</code> 。
<code>valueMissing</code>	如果元素（具有 <code>required</code> 属性）没有值，则设置为 <code>true</code> 。
<code>valid</code>	如果元素的值有效，则设置为 <code>true</code> 。

例如下

rangeOverflow属性

```
if (document.getElementById("id1").validity.rangeOverflow) {  
  text = "Value too large";  
}
```

rangeUnderflow属性

```
if (document.getElementById("id1").validity.rangeUnderflow) {  
  text = "Value too small";  
}
```

003.WebAPI访问windows.history对象：

windows.history对象包含用户访问过的所有URL网站

history.back () 方法：加载历史列表中的前一个URL同后退箭头效果一致

```
function myFunction () {  
  window.history.back ()  
}
```

history.go () 方法：加载历史列表中一个特定的URL，例子是后退两页

```
function myFunction () {  
  window.history.go (-2)  
}
```

history对象属性：

length，返回历史列表URL数量

history对象方法：

back () 加载历史列表中的上一个URL

forward () 加载历史列表中的下一个URL

go () 在历史列表中家在特定的URL

004.WebAPI访问浏览器的存储与检索数据

Web Storage API是一种在浏览器中存储和检索数据的简单语法

localStorage对象提供对特定网站的本地存储的访问，允许对存储、读取、添加、修改、删除该域的数据项，存储的数据没有到期日期，且浏览器关闭不会删除。

localStorage.setItem ()：方法用于将数据存储到storage中，仅接受一个名称与值作为参数

localStorage.setItem ("name", "value")

localStorage.getItem ()：方法用于检索数据项，仅接受名称作为参数

localStorage.getItem ("name")

sessionStorage对象与**localStorage**对象相同、但是**sessionStorage**对象会存储一个会话的数据，浏览器关闭删除

sessionStorage.getItem ()：从存储中检索数据项

sessionStorage.getItem ("name")

sessionStorage.setItem ()：将数据像存储在storage中

Storage对象的属性与方法：

key (n) 返回存储在storage中的第n个键的名称

length返回存储在storage对象中的数据长度

getItem (keyName) 返回指定的键名的值

setItem (key, value) 添加或更新该键值到storage存储中

removeItem (keyName) 从storage对象中删除该键

clear () 清空左右键

WebStorageAPI相关页面：

window.localStorage：允许在web中保存键值对、存储没有到期日期的数据

window.sessionStorage：允许在web中保存键值对、存储一个会话的数据

005.Web Worker API：

WebWorker在后台运行，不会影响页面的性能，独立于其他js脚本

检查WebWorker浏览器：

在创建WebWorker之前，请检查用户的浏览器是否支持

```

if (typeof(Worker) !== "undefined") {
    // Yes! Web worker support!
    // Some code.....
} else {
    // Sorry! No Web Worker support..
}

```

创建WebWorker文件：

在外部js中创建，通常WebWorker不是简单脚本，而是用于CPU的密集型任务

```

let i = 0;
function timedCount () {
    i ++;
    postMessage (i); //关键代码用于将消息发送给HTML页面
    setTimeout ("timedCount ()", 500);
}
timedCount ();

```

创建WebWorker对象：

创建完外部文件之后，需要从内部HTML页面调用

检查worker文件存在与否：不存在则新建一个对象并运行worker代码

```

if (typeof (w) == "undefined") {
    w = new Worker (".test.js")}

```

然后发送接收Web Worker的消息：为之添加一个事件侦听器

```

w.onmessage = function (event) {
    document.getElementById ("test") .innerHTML = event.data;
}

```

当Web Worker发布消息时，将执行事件侦听器中的代码，将Worker的数据存储在event.data文件中

终止WebWorker：终止worker，释放浏览器资源

w.terminate ()；方法终止w这个Worker

重用WebWorker：将worker变量设置为underfined，则在他终止后，可以重用以下代码

```

w = undefined;

```

WebWorker位于外部文件，无法访问以下对象

window对象

document对象

parent对象

006.获取API:

fetchAPI接口允许Web浏览器向Web服务器发出HTTP请求

FetchAPI示例：获取一个文件并显示其内容

fetch (file)

.then (x⇒x.text())

.then(y ⇒ myDisplay(y));

由于Fetch基于async和await，上下例等同

```
async function getText (file) {
```

```
  let x = await fetch (file);
```

```
  let y = await x.text ();
```

```
  myDisplay (y);
```

```
}
```

007.网络地理定位API:

定位用户的位置：需要用户批准，对于配给GPS的设备，地理定位最为准确

地理位置 API 仅在 HTTPS 等安全环境下有效。如果您的网站托管在非安全来源（例如 HTTP）上，则获取用户位置的请求将不再起作用。

使用地理位置API：getCurrentPosition () 方法用于返回用户的位置

下例返回用户的经纬度：

```
<script>
```

```
const x = document.getElementById("demo");
```

```
function getLocation() {
```

```
  if (navigator.geolocation) {
```

```
    navigator.geolocation.getCurrentPosition(showPosition);
```

```

} else {
x.innerHTML = "Geolocation is not supported by this browser.";
}
}

function showPosition(position) {
x.innerHTML = "Latitude: " + position.coords.latitude +
"<br>Longitude: " + position.coords.longitude;
}

</script>

```

处理错误和拒绝：`getCurrentPosition ()` 方法的第二个参数用于处理错误，它指定在无法获取用户位置时运行的函数。

```

function showError(error) {
switch(error.code) {
case error.PERMISSION_DENIED:
x.innerHTML = "User denied the request for Geolocation."
break;
case error.POSITION_UNAVAILABLE:
x.innerHTML = "Location information is unavailable."
break;
case error.TIMEOUT:
x.innerHTML = "The request to get user location timed out."
break;
case error.UNKNOWN_ERROR:
x.innerHTML = "An unknown error occurred."
break;
}
}
}

```

在地图中显示结果：需要访问地图服务例如Google地图等

下例返回经纬度用于显示Google地图中的位置

```
function showPosition(position) {  
    let latlon = position.coords.latitude + "," + position.coords.longitude;  
    let img_url = "https://maps.googleapis.com/maps/api/staticmap?center=  
    "+latlon+"&zoom=14&size=400×300&sensor=false&key=YOUR_KEY";  
    document.getElementById("mapholder").innerHTML = "<img  
    src='"+img_url+"'>";  
}
```

getCurrentPosition () 方法成功时返回一个对象，始终返回经纬度、准确度，如果可用、也会返回其他属性

coords.latitude：纬度，十进制，始终返回

coords.longitude：经度，十进制，始终返回

coords.accuracy：位置的精度，米，始终返回

coords.altitude：海拔高度，米，相对于海平面，可用则返回

coords.altitudeAccuracy：海拔高度的精度，米，可用则返回

coords.heading：方向，度°，从正北方向顺时针测量，可用则返回

coords.speed：速度，米每秒，可用则返回

timestamp：响应的时间和日期,可用则返回